

Combinatorial Optimization and Exact Methods

ENSIA Third Year -2026-

April, 27th

Outline

- 1 Introduction
- 2 Exact Methods
- 3 Enumeration
- 4 Branch and Bound
- 5 Example: Integer Linear Programming
- 6 Assignment Problem Example
- 7 TSP and B&B

Definition:

A combinatorial optimization problem consists of:

$$\min_{s \in S} f(s) \quad \text{or} \quad \max_{s \in S} f(s)$$

- S : finite set of feasible solutions
- $f(s)$: objective function

Definition:

A combinatorial optimization problem consists of:

$$\min_{s \in S} f(s) \quad \text{or} \quad \max_{s \in S} f(s)$$

- S : finite set of feasible solutions
- $f(s)$: objective function

Characteristics:

- Discrete decision variables
- Large search space (often exponential)
- Many problems are NP-hard

Examples

- Assignment Problem
- Traveling Salesman Problem (TSP)
- Knapsack Problem
- Scheduling Problems

Goal: Find the optimal solution with proof of optimality

Goal: Find the optimal solution with proof of optimality

Main approaches:

- Complete Enumeration
- Dynamic Programming
- Cutting Plane Methods
- Branch and Bound

Goal: Find the optimal solution with proof of optimality

Main approaches:

- Complete Enumeration
- Dynamic Programming
- Cutting Plane Methods
- Branch and Bound

Limitation:

- High computational complexity

Explicit Enumeration

- Evaluate all feasible solutions
- Example: Assignment problem ($n!$ solutions)

Explicit Enumeration

- Evaluate all feasible solutions
- Example: Assignment problem ($n!$ solutions)

Drawback:

- Not scalable

Implicit Enumeration

- Avoid exploring all solutions
- Use bounds to eliminate subsets

Implicit Enumeration

- Avoid exploring all solutions
- Use bounds to eliminate subsets

Key Idea:

- Reduce search space intelligently

Implicit Enumeration

- Avoid exploring all solutions
- Use bounds to eliminate subsets

Key Idea:

- Reduce search space intelligently

Leads to Branch and Bound

Branch and Bound Principle

Three main components:

- **Branching**: divide problem into subproblems
- **Bounding**: compute bounds on solutions
- **Pruning**: eliminate non-promising subproblems

General Optimization Model

$$\min_{s \in S} f(s)$$

$$\min_{s \in S} f(s)$$

Evaluation:

For a subset $S' \subseteq S$, compute a bound $e(S')$ such that:

$$e(S') \leq f(s), \quad \forall s \in S'$$

Evaluation:

$e(S')$ is a bound on solutions in S'

Evaluation:

$e(S')$ is a bound on solutions in S'

Exact Evaluation:

$$\exists \hat{s} \in S' : f(\hat{s}) = e(S')$$

Definitions (from PDF)

Evaluation:

$e(S')$ is a bound on solutions in S'

Exact Evaluation:

$$\exists \hat{s} \in S' : f(\hat{s}) = e(S')$$

Sterile Set:

S' is **sterile** if:

$$f(s^*) \leq e(S')$$

where s^* is the best known solution

Definitions (from PDF)

Evaluation:

$e(S')$ is a bound on solutions in S'

Exact Evaluation:

$$\exists \hat{s} \in S' : f(\hat{s}) = e(S')$$

Sterile Set:

S' is **sterile** if:

$$f(s^*) \leq e(S')$$

where s^* is the best known solution

Implication: No need to explore S'

Branching (Separation)

A set S' is divided into subsets:

$$S' = \bigcup_{i=1}^k S'_i$$

Branching (Separation)

A set S' is divided into subsets:

$$S' = \bigcup_{i=1}^k S'_i$$

Goal:

- Explore smaller subproblems

Key Theorem

Let:

$$S = \bigcup_{i=1}^k S_i$$

Then:

$$\min_{s \in S} f(s) = \min_{i=1, \dots, k} \left(\min_{s \in S_i} f(s) \right)$$

Key Theorem

Let:

$$S = \bigcup_{i=1}^k S_i$$

Then:

$$\min_{s \in S} f(s) = \min_{i=1, \dots, k} \left(\min_{s \in S_i} f(s) \right)$$

Meaning:

- Solve subproblems instead of full problem

Initialization:

- $F = \{S\}$
- Best value $V = +\infty$

Initialization:

- $F = \{S\}$
- Best value $V = +\infty$

Loop:

- 1 Select subset S'
- 2 Compute $e(S')$
- 3 If S' is sterile $\rightarrow$ prune
- 4 Else branch into subsets

- **Best-first search** (smallest bound)
- **Depth-first search**
- **Breadth-first search**

- **Best-first search** (smallest bound)
- **Depth-first search**
- **Breadth-first search**

Practical approach:

- Hybrid strategies

Example (Relaxation)

Solve LP relaxation:

$$\max Z = 2x_1 + x_2$$

subject to constraints

Example (Relaxation)

Solve LP relaxation:

$$\max Z = 2x_1 + x_2$$

subject to constraints

Observation:

- Solution may not be integer

If $x_1 = 3.75$:

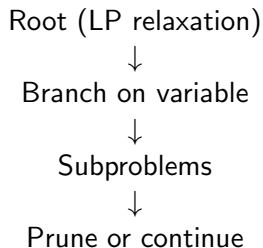
- Branch 1: $x_1 \leq 3$
- Branch 2: $x_1 \geq 4$

If $x_1 = 3.75$:

- Branch 1: $x_1 \leq 3$
- Branch 2: $x_1 \geq 4$

Solve each subproblem

Tree Representation



Assignment Problem

$$\min \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

subject to:

$$\sum_j x_{ij} = 1, \quad \sum_i x_{ij} = 1$$

$$x_{ij} \in \{0, 1\}$$

Idea:

Sum of minimum values of each column:

$$\sum_{j=1}^n \min_i c_{ij}$$

Idea:

Sum of minimum values of each column:

$$\sum_{j=1}^n \min_i c_{ij}$$

Property:

- This is a lower bound of any feasible solution

TSP Relaxation

- Use Assignment Problem as relaxation
- Ignore connectivity constraint

TSP Relaxation

- Use Assignment Problem as relaxation
- Ignore connectivity constraint

Result:

- Provides lower bound

1-tree Concept

Definition:

- Tree on nodes $\{2, \dots, n\}$
- Add two edges connected to node 1

1-tree Concept

Definition:

- Tree on nodes $\{2, \dots, n\}$
- Add two edges connected to node 1

Property:

- Any TSP tour is a 1-tree

Example

An administrator wants to assign four workers to four tasks such that:

- Each task is performed by only one worker.
- Each worker performs only one task.

The following matrix gives the assignment costs of each worker to each task:

$$C = \begin{array}{c|cccc} & T_1 & T_2 & T_3 & T_4 \\ \hline O_1 & \mathbf{1} & 4 & 6 & 3 \\ O_2 & 8 & \mathbf{7} & 10 & 9 \\ O_3 & 4 & 5 & \mathbf{11} & 7 \\ O_4 & 4 & 7 & 8 & \mathbf{5} \end{array}$$

Example of an Assignment

$$T_1 \rightarrow O_1 \quad T_2 \rightarrow O_2 \quad T_3 \rightarrow O_3 \quad T_4 \rightarrow O_4$$

Assignment $\equiv$ Permutation

Explicit Enumeration Method

One way to solve the problem is to enumerate all possible assignments:

$$n! \quad \text{with } n = 4$$

and choose the one with minimum cost.

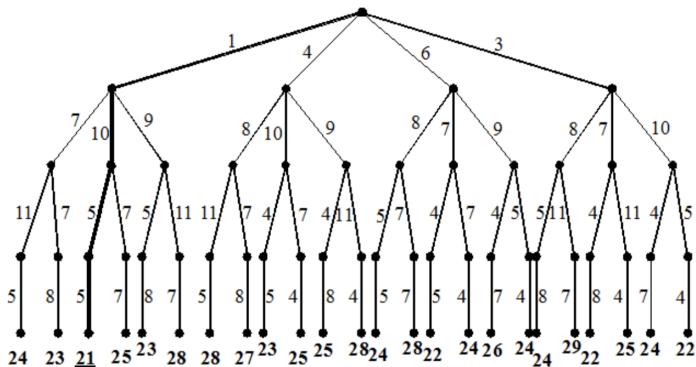
This method is called:

Explicit Enumeration of Assignments

Construction of Assignments

Assignments can be built step by step:

- At the first step, assign worker O_1 to one of the n tasks.
- At the second step, assign worker O_2 to one of the remaining $(n - 1)$ tasks.
- Continue in the same way until all workers are assigned.



Implicit Enumeration for Assignment Problem

Idea: Construct a search tree using a **lower bound**.

Implicit Enumeration for Assignment Problem

Idea: Construct a search tree using a **lower bound**.

- Start with a lower bound of the cost of any assignment
- Subtract the minimum of each column
- Try to find a feasible assignment

Implicit Enumeration for Assignment Problem

Idea: Construct a search tree using a **lower bound**.

- Start with a lower bound of the cost of any assignment
- Subtract the minimum of each column
- Try to find a feasible assignment

Two cases:

- If an assignment is found → **stop branching (prune / sterilize)**
- Otherwise → **branch** by assigning tasks to a worker

Implicit Enumeration for Assignment Problem

Idea: Construct a search tree using a **lower bound**.

- Start with a lower bound of the cost of any assignment
- Subtract the minimum of each column
- Try to find a feasible assignment

Two cases:

- If an assignment is found → **stop branching (prune / sterilize)**
- Otherwise → **branch** by assigning tasks to a worker
- Recompute the lower bound at each node
- Continue until all subsets are **sterilized**

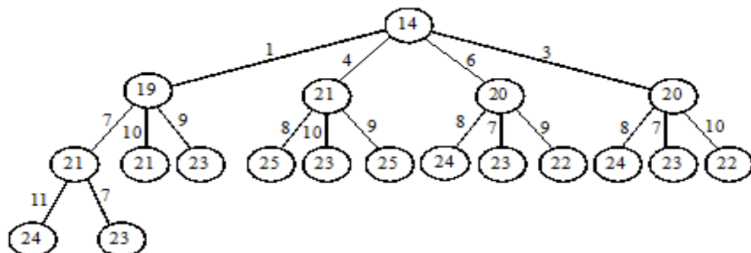
Example:

- Consider the first worker
- Assign progressively tasks T_i , $i = 1, \dots, 4$

Example:

- Consider the first worker
- Assign progressively tasks T_i , $i = 1, \dots, 4$
- Each assignment creates a node in the tree
- Each node corresponds to a partial solution

Branch and Bound Tree (Assignment)



Example of implicit enumeration tree with bounds

Stronger Bound:

Improved Lower Bound

Stronger Bound:

For a cost matrix C :

Lower Bound

- Sum of column minima
- + Sum of row minima of reduced matrix C'

Improved Lower Bound

Stronger Bound:

For a cost matrix C :

Lower Bound

- Sum of column minima
- + Sum of row minima of reduced matrix C'

Where:

- C' is obtained by subtracting column (or row) minima from C

Improved Lower Bound

Stronger Bound:

For a cost matrix C :

Lower Bound

- Sum of column minima
- + Sum of row minima of reduced matrix C'

Where:

- C' is obtained by subtracting column (or row) minima from C

Property:

- This gives a tighter lower bound $\rightarrow$ better pruning